

Protokoll: myfind (Parallele Dateisuche)

Antonios Sotiriou & Alev Karadeniz, 25.09.2026

1. Beschreibung der Architektur und Parallelisierung:

Dieses Projekt implementiert eine parallele Dateisuche in C. Anstatt die Dateien nacheinander zu suchen, sucht das Programm mehrere Dateien gleichzeitig.

Prozess-Erstellung (fork): Das Hauptprogramm durchläuft die Liste der gesuchten Dateinamen. Für jeden Dateinamen wird mit `fork()` ein eigener Kind-Prozess (Child-Process) erstellt. Wenn der Benutzer nach 3 Dateien sucht, arbeiten 3 Kind-Prozesse gleichzeitig.

Warten auf Kinder (wait): Der Eltern-Prozess (Parent-Process) darf nicht beendet werden, bevor die Kinder fertig sind. Mit einer `while`-Schleife und der Funktion `wait()` wartet der Parent auf alle Kinder. Die Rückgabewerte (Exit-Codes) werden mit `WIFEXITED` geprüft. Dadurch werden verwaiste (Zombie) Prozesse verhindert.

Output-Synchronisation: Da viele Prozesse gleichzeitig laufen, könnten sich die Textausgaben auf dem Bildschirm vermischen. Um das zu verhindern, wird die gesamte Ausgabe (PID, Dateiname, absoluter Pfad) in einem einzigen, atomaren `fprintf()`-Aufruf mit einem Zeilenumbruch (`\n`) gebündelt. Das Betriebssystem schreibt diese kleinen Blöcke sicher in die Konsole, sodass die Zeilen unsortiert, aber immer komplett lesbar bleiben.

2. Beschreibung der Suchlogik und Argumenten-Verwaltung

Argumente (getopt): Die Funktion `parseArguments` nutzt `getopt()`, um die Flags `-R` (rekursiv) und `-i` (Groß-/Kleinschreibung ignorieren) zu finden, egal an welcher Stelle sie eingegeben werden. Die Suchpfade und Dateinamen werden dynamisch mit `malloc` im Speicher gesichert.

Ordner durchsuchen (searchFolder): Mit `opendir()` und `readdir()` werden die Ordner gelesen. Die Einträge `."` und `".."` werden ignoriert, um Endlosschleifen zu vermeiden.

Rekursion (-R): Wenn das Flag `-R` aktiv ist und ein Unterordner (`DT_DIR`) gefunden wird, ruft sich die Funktion `searchFolder` rekursiv selbst auf.

Dateivergleich (-i): Normale Suchen nutzen `strcmp()`. Ist `-i` aktiv, wird `strcasecmp()` verwendet. Wenn eine Datei gefunden wird, wandelt die Funktion `realpath()` den relativen Pfad (z.B. `./test.c`) in den geforderten absoluten Pfad (z.B. `/home/user/test.c`) um.